A lambda is a function you write right where you need it.

```
std::sort(v.begin(), v.end(),
          [](int a, int b) {
              return a > b;
          });
```

No separate function. No name. Just the logic, in place.

swipe →

Three parts

```
[limit](int x) {
    return x > limit;
}
```

WHAT IT IS	
`[ ]`	the **capture**: what it can see from outside
`( )`	the **parameters**, like any function
`{ }`	the **body**

The capture list

WRITE	MEANING
`[]`	nothing from outside
`[x]`	a **copy** of `x`
`[&x]`	a **reference** to `x`
`[=]`	copy everything it uses
`[&]`	reference everything it uses

Listing names explicitly makes it obvious what the lambda depends on.

Where it shines

```
int limit = 3;

auto n = std::count_if(
    v.begin(), v.end(),
    [limit](int x) { return x > limit; });
```

The standard algorithms take a **rule** as an argument.

I ran this on `{3, 1, 4, 1, 5, 9, 2, 6}`: it counted 4 numbers above 3.

The trap: a reference that outlives

```
auto make() {
    int n = 5;
    return [&n] { return n; };   // n is gone
}
```

The lambda now holds a dangling reference.

Fix: capture by value, `[n]`.

Capture by reference only when the lambda is used and finished while the variable is still alive.

It is really an object

```
auto twice = [](int x) {
    return x * 2;
};
```

The compiler writes a small class with an `operator()`.

- `auto` keeps its exact type: fast, can be inlined
- `std::function` can hold any callable, with some overhead

Do you write `[=]` or list each capture?